

Izgradnja i treniranje neuronske mreže



Sadržaj

- Izgradnja i treniranje neuronske mreže
- Pozivne funkcije (*callbacks*) u Kerasu

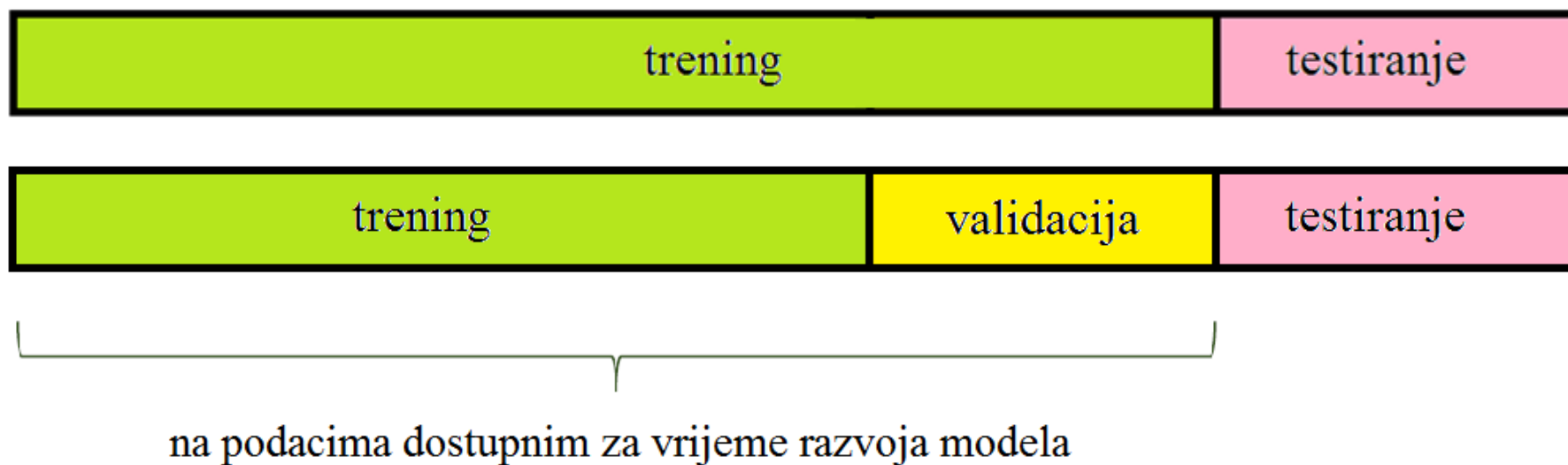
Izgradnja i treniranje neuronske mreže

Izgradnja neuronske mreže

- Izgradnja neuronske mreže podrazumijeva različite korake
 - **Priprema skupova podataka za učenje, validaciju i testiranje**
 - **Predobrada podataka – što sve može uključivati?**
 - Odabir strukture neuronske mreže (slojevi, broj neurona, aktivacijske funkcije)
 - Inicijalizacija parametara mreže
 - **Kriterijska funkcija (*loss function*)**
 - Regularizacija (promjena kriterijske funkcije (L2 i L1 regularizacija), dropout, ...)
 - *Batch* normalizacija
 - Podešavanje različitih hiperparametara
 - **Stope učenja**
 - **Mijenjanje strukture mreže,**
 - **Veličine *batch*-a,**
 - Parametra regularizacije, ...
 - ...

Priprema skupova podataka za učenje, validaciju i testiranje

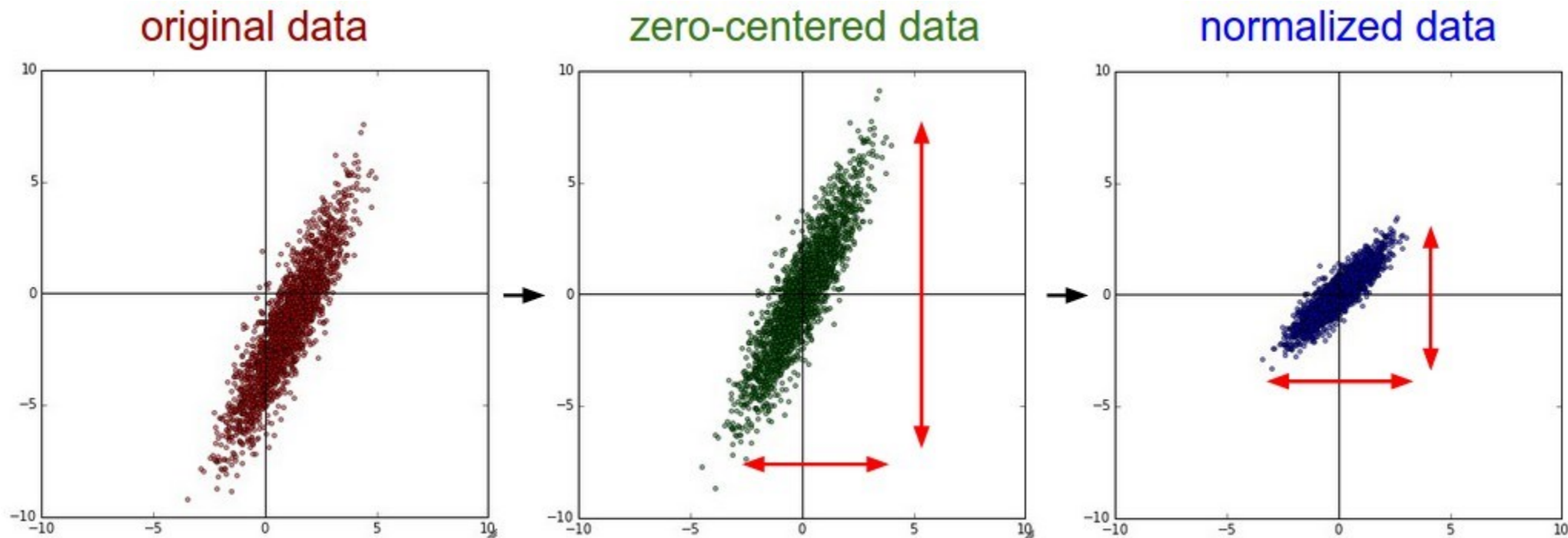
- Ako se želi izvjesiti o pogrešci, odnosno kolika je očekivana pogreška najboljeg modela, treba se koristiti skup podataka koji nije korišten za vrijeme treninga – testni skup podataka
- Naime, budući da je validacijski skup podataka korišten za odabir najboljeg modela, on je time efektivno postao dio skupa za učenje



Predobrada podataka

- Što sve može uključivati postupak predobrade podataka?
- Što je skaliranje podataka i kada ga je potrebno raditi?
- Što je smanjivanje dimenzionalnosti podataka i zašto se radi?
- Što znači da je skup podataka balansiran?
- Što je augmentacija skupa podataka?

Predobrada podataka



- Kod slika samo se centriraju podaci (efikasniji postupak optimizacije parametara)
- Obično se ne radi se skaliranje (jer pikseli imaju istu “dimenziju”, npr. tri kanala 0-255)

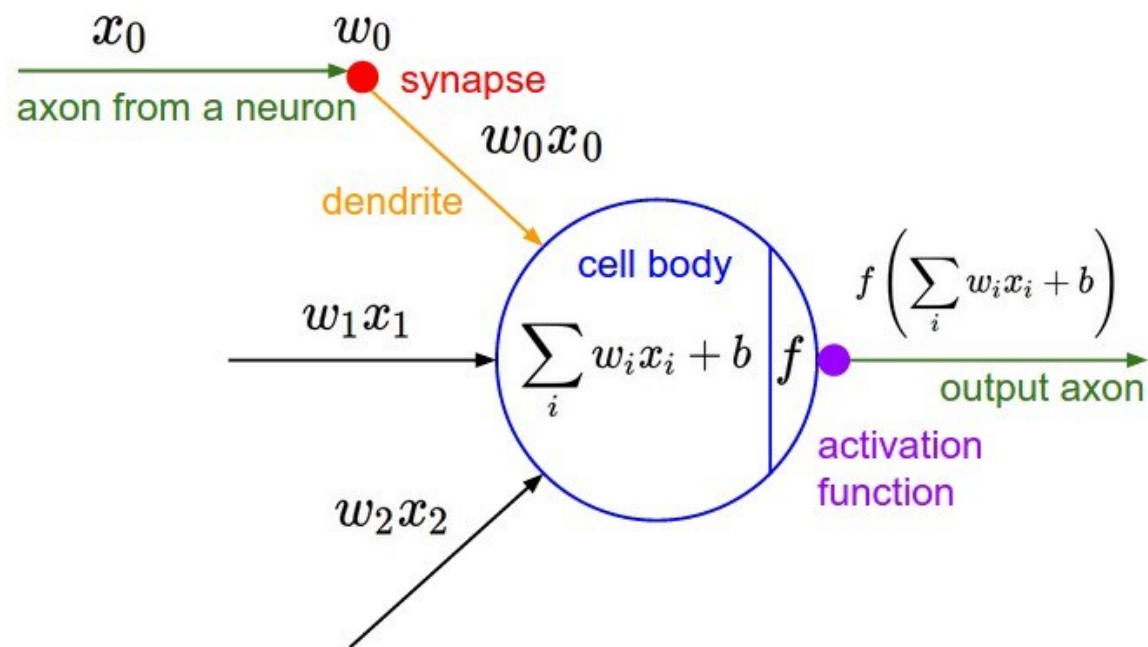
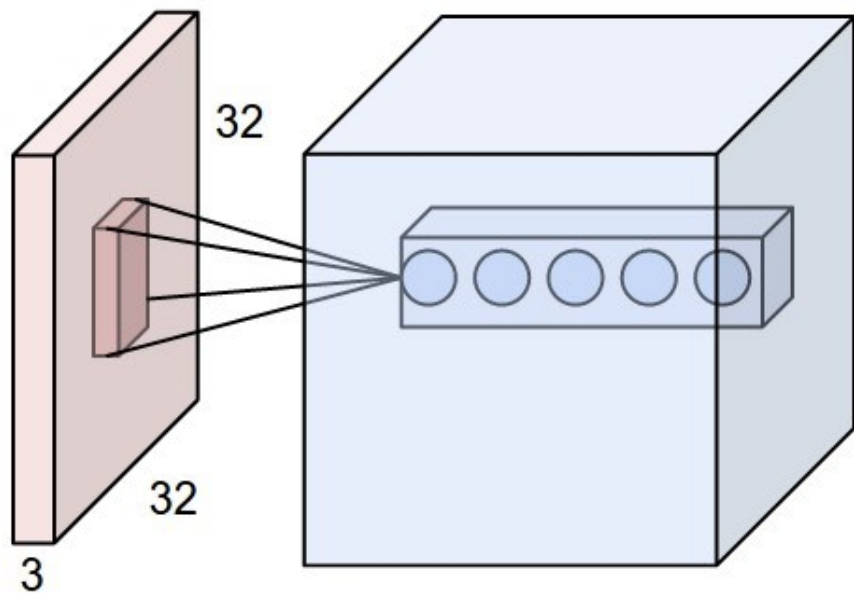
Odabir strukture i veličine neuronske mreže

- Odabir veličine mreže, bilo da se radi o tradicionalnim (potpuno povezanim) NN ili CNN, uvijek je bio izazovan problem
- Npr., broj težinskih parametara (težina) i broj slojeva moraju se podesiti kako bi se postigle tražene performanse
- Npr. jednostavna mreža bez ijednog skrivenog sloja mogla bi dati samo linearnu granicu odluke, što nije dovoljno za rješavanje isključivog ili (XOR) ili sličnog problema
- Kapacitet mreže odnosi se na razinu složenosti funkcije koju ona može naučiti aproksimirati
- Male mreže ili mreže s relativno malim brojem parametara imaju nizak kapacitet → vjerojatno će se nedovoljno prilagoditi (*underfitting*), što će rezultirati lošim performansama, budući da ne mogu naučiti osnovnu strukturu složenih skupova podataka

Odabir strukture i veličine neuronske mreže

- S druge strane, vrlo velike mreže mogu dovesti do prekomjernog prilagođavanja (*overfitting*), gdje će mreža pamtiiti trening podatke i postizati na trening skupu visoke performanse, dok će postići loše performanse na validacijskom, a i na testnom skupu podataka
- Kada se bavimo problemima strojnog učenja u stvarnom svijetu, ne znamo unaprijed kolika bi mreža trebala biti
 - Jedan od načina za rješavanje ovog problema je izgradnja mreže s relativno velikim kapacitetom (u praksi, želimo odabrati kapacitet koji je malo veći od potrebnog) kako bi se dobro snašla na skupu trening podataka (kako bi se spriječio *underfitting*)
 - Zatim, kako bismo spriječili prekomjerno prilagođavanje na trening podatke, **možemo primijeniti jednu ili više shema regularizacije kako bismo postigli željenu razinu generalizacije na novim podacima**

Odabir aktivacijske funkcije

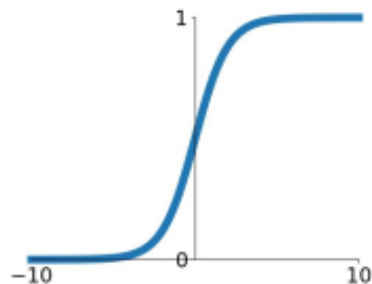


[Izvor](#)

Odabir aktivacijske funkcije

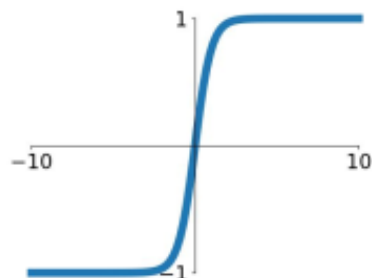
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



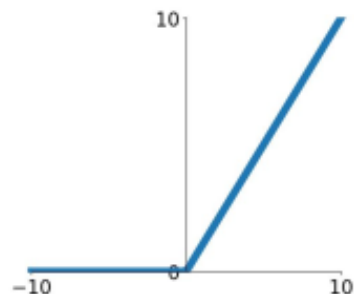
tanh

$$\tanh(x)$$



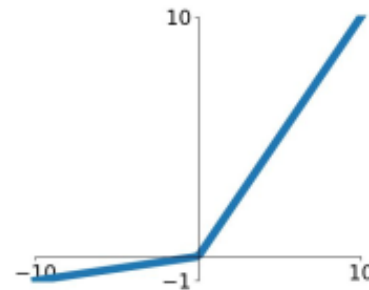
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

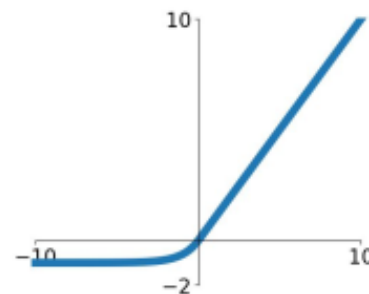


Maxout

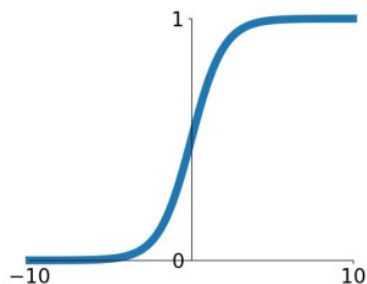
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

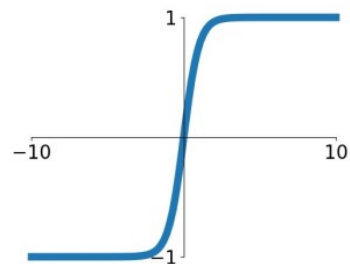


Odabir aktivacijske funkcije



Sigmoid

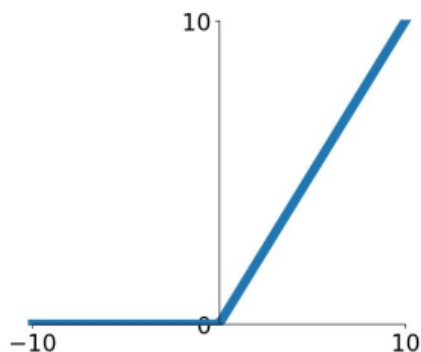
- Izlaz je u rasponu $[0,1]$
- Popularna iz povijesnih razloga
- **Problemi:**
 - Gradijent je 0 kada je neuron u zasićenju
 - Izlaz nije centriran oko 0 → uzrokuje neefikasnu procjenu optimalnih parametara (sporija konvergencija)
 - Funkcija $\exp$ je računalno zahtjevnija (npr. zahtjevnija od množenja)



tanh(x)

- Iako centrirana oko 0 i dalje postoji problem da nestaje gradijent kada je neuron u zasićenju

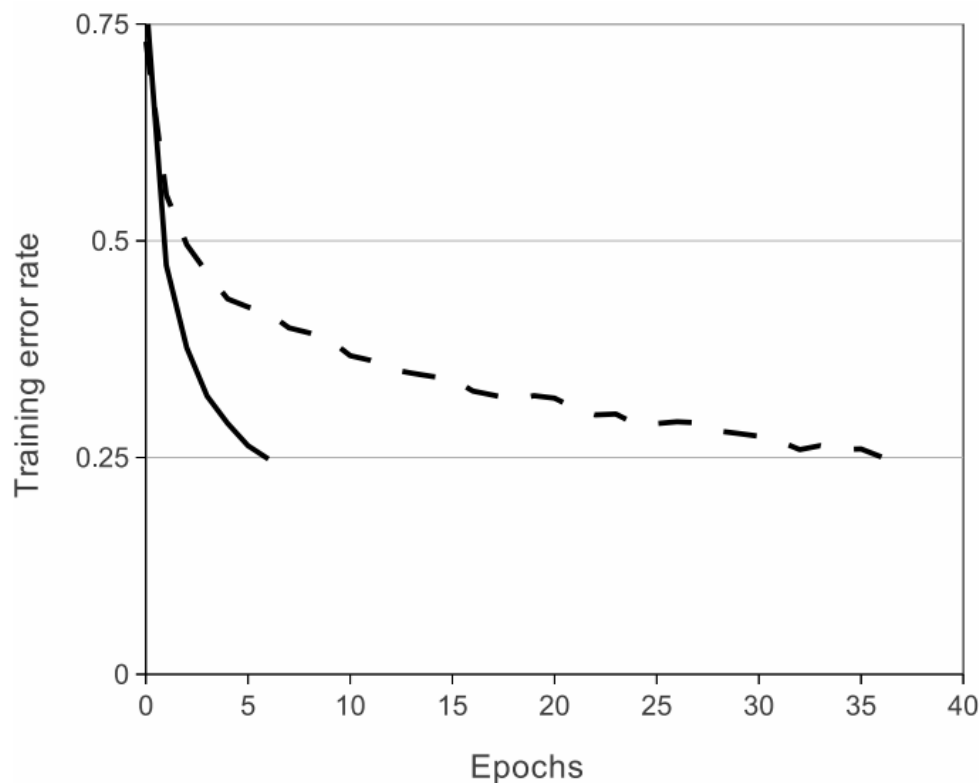
Odabir aktivacijske funkcije



ReLU
(Rectified Linear Unit)

- $f(x) = [0, \max(x)]$
- Nema problema s nestajanjem gradijenta
- Računalno efikasna
- U praksi znanto brže mreža konvergira od sigmoidnih ili tanh funkcija (važno kod velikih skupova za učenje)
- Dobro oponaša biološku neuronsku aktivnost
- **Problemi:**
 - Nije centrirana oko 0
 - U negativnom dijelu je gradijent 0
- Mrtvi ReLU → nikada se ne aktiviraju; ne osvježavaju se tijekom učenja
- **Praksa - često se ovo dogodi kada je prevelika stopa učenja; čak do 40% mreže može biti "mrtvo"**

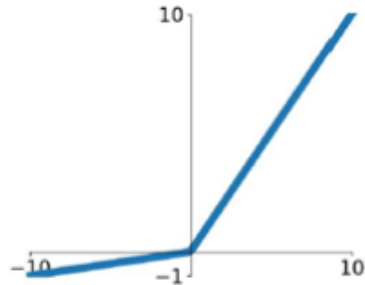
Odabir aktivacijske funkcije



- CNN sa 4 konvolucijska sloja koja koriste ReLU (**puna linija**) dostigla je na trening skupu grešku od 25% na CIFAR-10 šest puta brže nego ista mreža koja je koristila tanh aktivacijsku funkciju (**isprekidana linija**)
- Stope učenja za svaku mrežu bile su odabrane neovisno kako bi se trening obavio najbrže moguće
- Nije korištena regularizacija
- Ove razlike variraju s promjenom arhitekture mreže, **ali mreže s ReLU aktivacijskom funkcijom konzistentno uče nekoliko puta brže nego ekvivalentne mreže s neuronima koji ulaze u zasićenje**

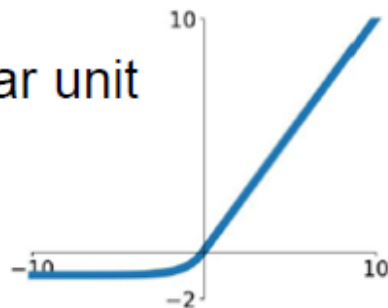
Odabir aktivacijske funkcije

Leaky ReLU
 $\max(0.1x, x)$



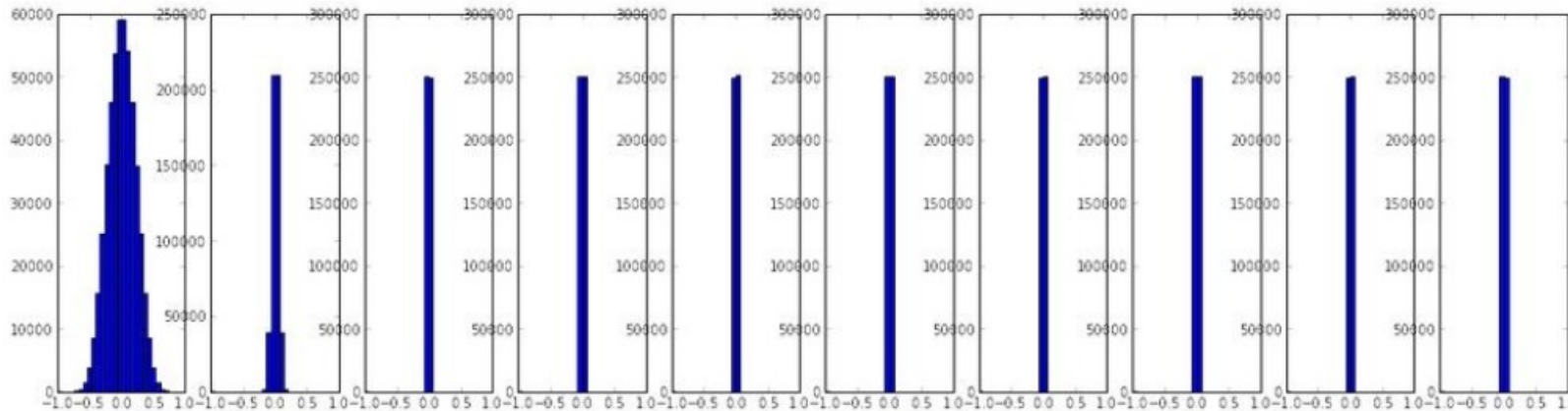
- Rješavaju u određenim slučajevima neke od problema koji postoje kod korištenja ReLU aktivacijske funkcije

ELU exponential linear unit

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$


Inicijalizacija parametara mreže

- Za plitke mreže uobičajeno je inicijalizirati parametre mreže da imaju srednju vrijednost 0 i neku malu standardnu devijaciju (npr. 0.01)
- Međutim, kod dubokih mreža ovakav način uzrokuje probleme prilikom optimizacije
- **Primjer:** mreža od 10 slojeva, 500 neurona u svakom sloju, s *tanh* aktivacijskim funkcijama



Propustimo nasumični podatak kroz mrežu (*forward pass*)

Statistika izlaza iz neurona (aktivacija) za svaki sloj

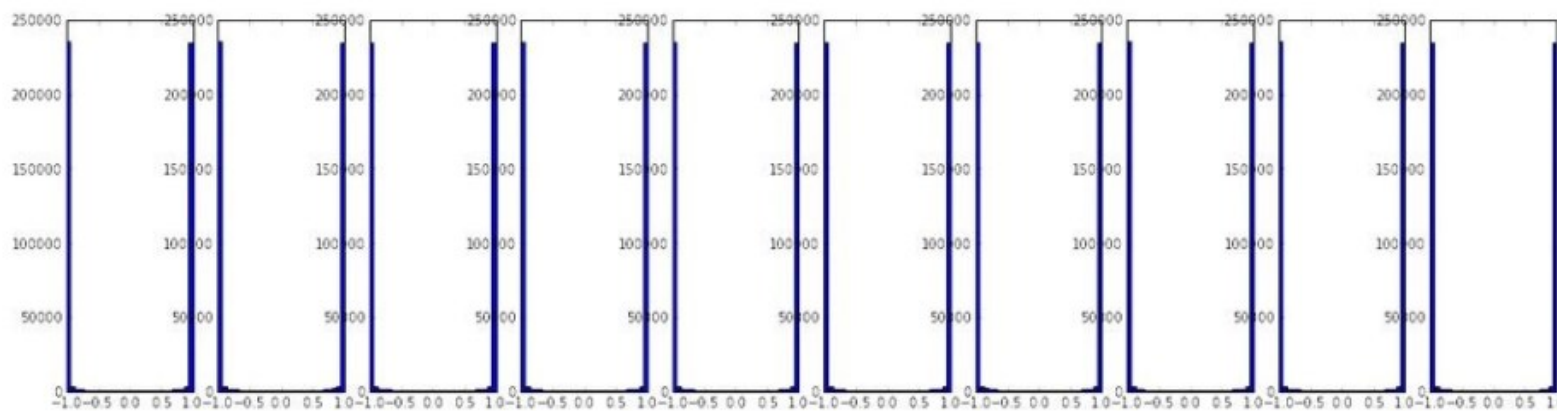
Brzo se smanjuje devijacija izlaza (teže u 0)

Backward pass: ovisi o vrijednostima ulaza za svaki neuron → zbog malih vrijednosti sporo se podešavaju parametri (gradijent je jako mali, engl. **vanishing gradient problem**)

Uzrok: množenje malih brojeva

Inicijalizacija parametara mreže

- **Pokušaj rješavanja problema**: inicijalizirati parametre mreže da imaju srednju vrijednost 0 i veću standardnu devijaciju (npr. 1) kako bi se izbjeglo “smanjivanje” kroz slojeve



Propustimo nasumični podatak kroz mrežu (*forward pass*)

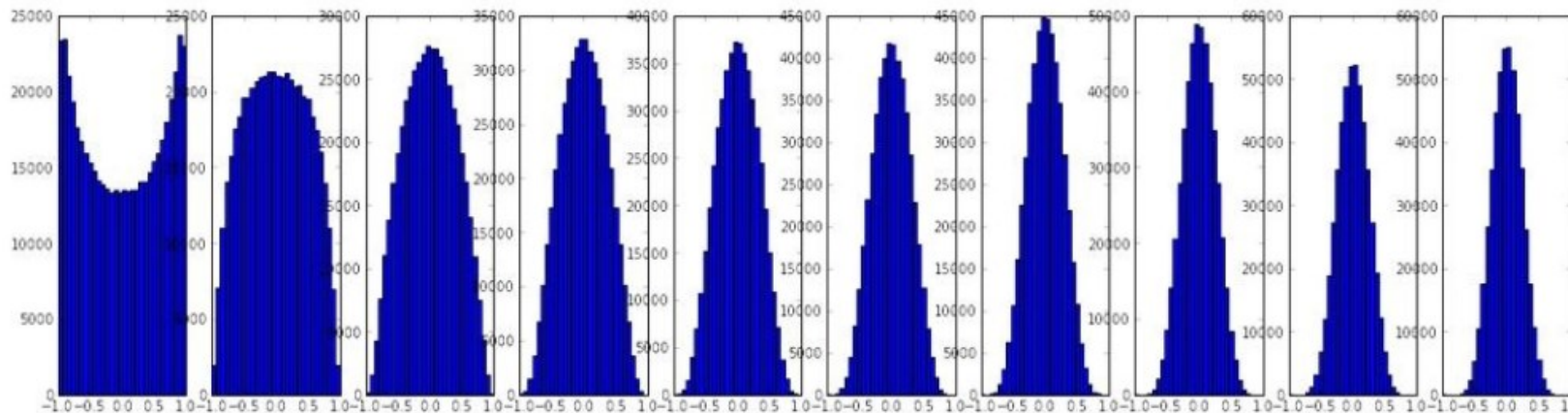
Statistika izlaza iz neurona (aktivacija) za svaki sloj

Neuroni su u zasićenju (-1 ili +1)

Gradijenti su jednaki 0

Inicijalizacija parametara mreže

- X.Glorot, Y.Bengio, [Understanding the difficulty of training deep feedforward neural networks](#), 2010.
- **Pravilna inicijalizacija mreže je trenutno aktivno područje istraživanje; različiti načini**
- Dobar pristup: **“varijanca ulaza treba biti jednaka varijanci izlaza”**



Tzv. Xavier pristup

Just right: nice distribution of activations at all layers → adaptation (learning) effective.

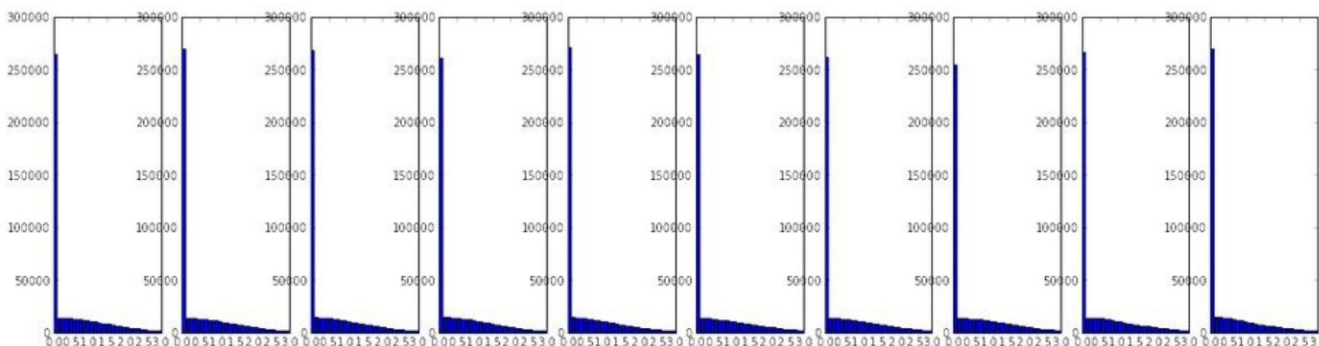
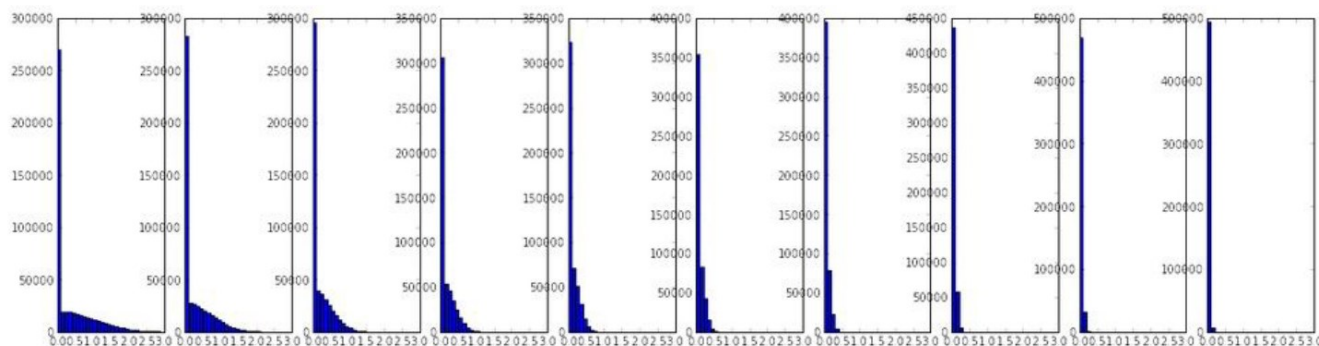
Prikladan način za *tanh* aktivacijske funkcije

$$\sqrt{\frac{1}{size^{[l-1]}}}$$

$$W^{[l]} = np.random.randn(size_l, size_l-1) * np.sqrt(1/size_l-1)$$

Inicijalizacija parametara mreže

- U slučaju **ReLU** „Xavier” pristup nije prikladan
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, [Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification](#), 2015.



In the following, we derive a theoretically more sound initialization by taking ReLU/PRelu into account. In our experiments, our initialization method allows for extremely deep models (*e.g.*, 30 conv/fc layers) to converge, while the “Xavier” method [7] cannot.

$$W^{[l]} = \text{np.random.randn}(\text{size}_l, \text{size}_{l-1}) * \text{np.sqrt}\left(\frac{2}{\text{size}_{l-1}}\right)$$

Treniranje neuronske mreže

- **Postupak učenja mreže**

1. Neuronska mreža

$$a_L(x; w_{1,...,L}) = h_L(h_{L-1}(...h_1(x, w_1), w_{L-1}), w_L)$$

2. Učenje minimizacijom empirijske pogreške

$$w^* \leftarrow \arg \min_w \sum_{(x,y) \in (X,Y)} \mathcal{L}(y, a_L(x; w_{1,...,L}))$$

3. Optimizacija parametara koristeći određenu gradijentnu metodu

$$w_{t+1} = w_t - \eta_t \nabla_w \mathcal{L}$$

Gradijentna metoda za treniranje neuronske mreže

- Za optimizaciju kriterijske funkcije koriste se gradijentne metode i njihove varijante
- Postoji razlika u odnosu na standardnu optimizaciju → bitno je kako će model funkcionirati u praksi!
- Ako se gradijent izračunava na temelju cjelokupnog skupa podataka učenja → ***batch gradient descent***
- **Prednosti *batch* metode:**
 - Konvergencija je dobro shvaćena (teorijski)
 - Mogu se koristiti različite tehnike ubrzanja
- **Nedostaci *batch* metode:**
 - Skupovi podataka često su vrlo veliki, pa je teško izračunati gradijente; spora konvergencija
 - Prikladno za konveksne, glatke kriterijske funkcije
 - To je također aproksimacija stvarnog gradijenta (jer imamo dostupan konačan skup podataka); optimalan za podatke koje imamo, ali ne mora značiti i za distribuciju koja ih je generirala

Gradijentna metoda za treniranje neuronske mreže

- Ako se težine prepodešavaju na temelju samo jednog podatka → ***stochastic gradient descend*** metoda
- U dubokom učenju koristi se ***minibatch stochastic gradient descend*** metoda optimizacije parametara → uzorkovanje mini-skupova za učenje
 - Isto kao i *batch* metoda, ovo je aproksimacija stvarnog gradijenta;
 - Međutim, sadrži šum koji je vrlo poželjan u nekonveksnoj optimizaciji (algoritam može izbjeći lokalne minimume i sedla te sprječava *overfitting*)
 - Prikladnije s obzirom na računalne zahtjeve; preporuka veličine → isprobajte 32
 - Pomaže generalizaciji (pretražuje "ravne" minimume)
- [Dominic Masters, Carlo Luschi, Revisiting Small Batch Training for Deep Neural Networks, 2018.](#)

Gradijentna metoda za treniranje neuronske mreže

- I. Goodfellow et. al. [Deep Learning](#), 2016.
- Stopa učenja se smanjuje tijekom iteracija – što se time dobiva?

Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

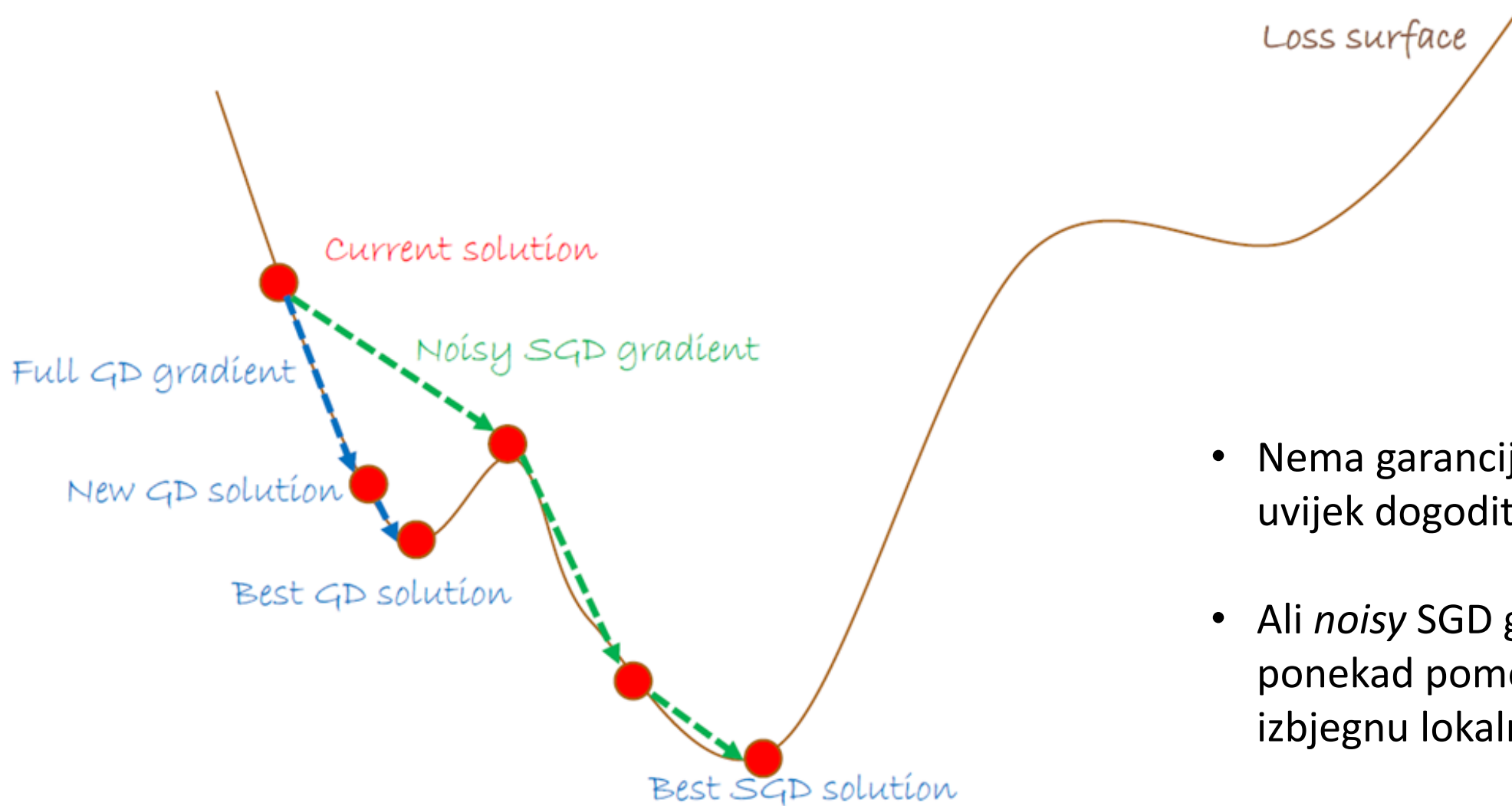
$k \leftarrow k + 1$

end while

Različite metode optimizacije

- SGD (stochastic gradient descend)
 - SGD s momentum
 - Adagrad
 - Adadelata
 - RMSprop
 - Adam...
-
- Više informacija na:
<https://runder.io/optimizing-gradient-descent/>

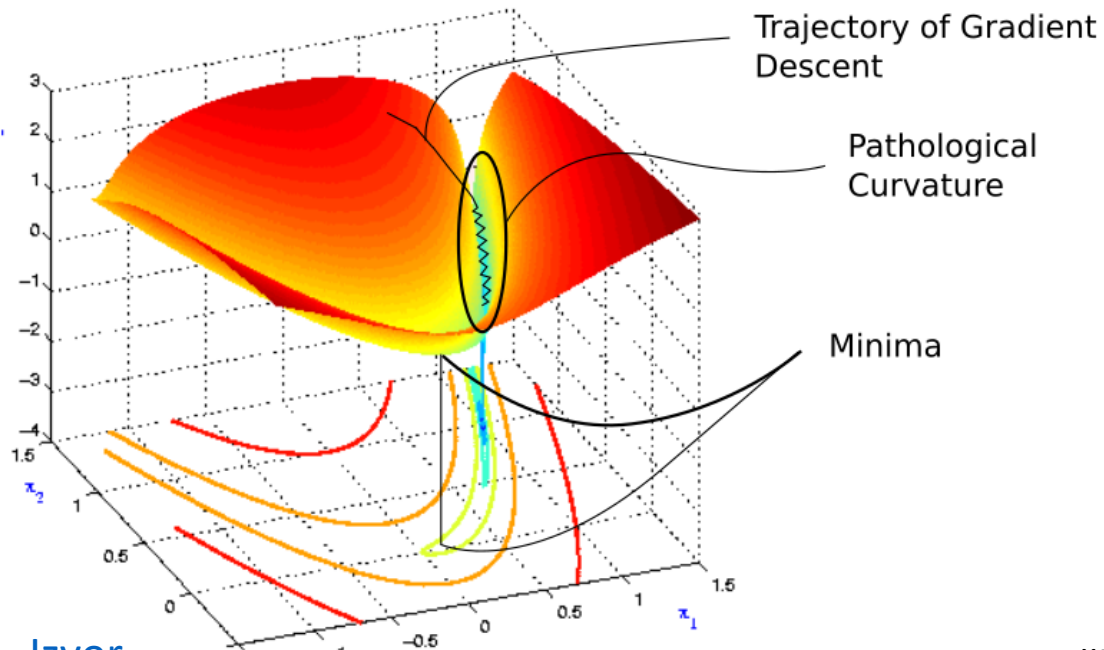
SGD može često pomoći...



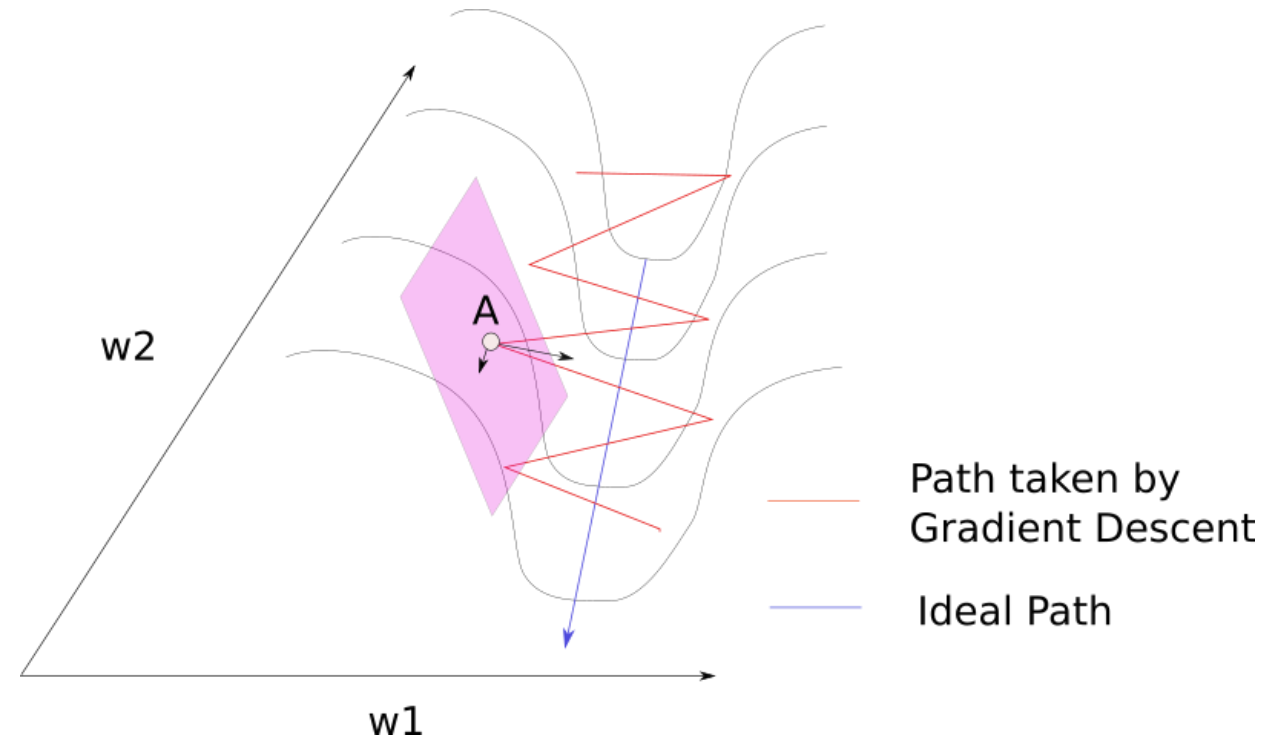
- Nema garancije da će se ovo uvijek dogoditi
- Ali *noisy* SGD gradijent može ponekad pomoći da se izbjegnu lokalni minimumi

Optimizacija SGD – problem 1

- Uvala - vrlo spor progres tijekom jedne dimenzije, zig-zag tijekom druge dimenzije
- Ovaj problem je čest kod visokodimenzionalnih problema
- Može se smanjiti stopu učenja → vrlo spora konvergencija



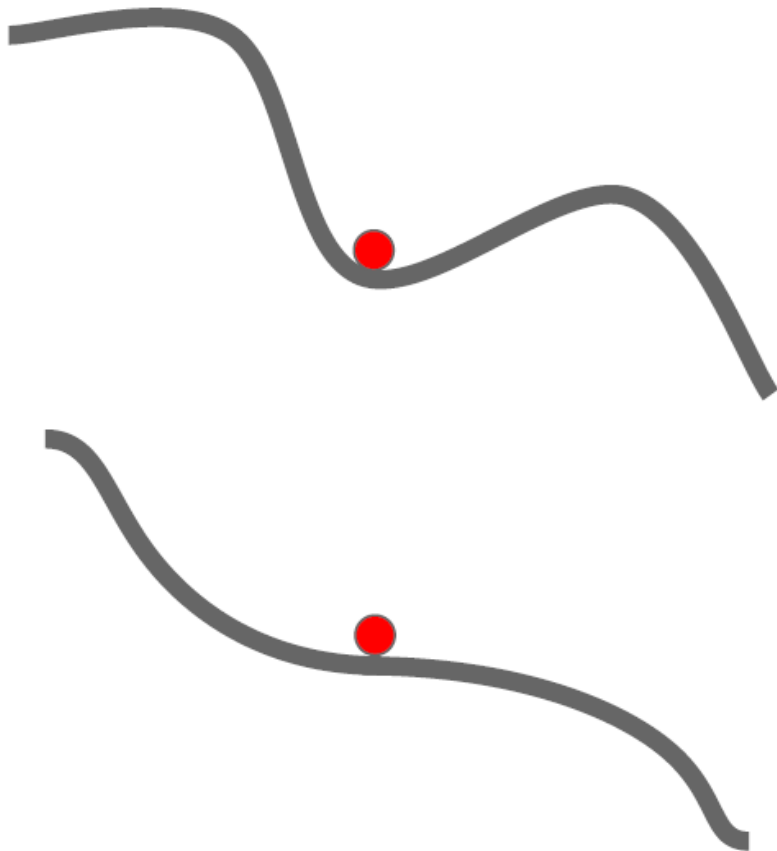
[Izvor](#)



Točka A: Gradijent u smjeru w_1 znatno je veći nego u smjeru w_2

Optimizacija SGD – problem 2

- Lokalni minimumi i sedla kriterijske funkcije



- Lokalni minimum $\rightarrow$ Gradijent je jednak 0
- Sedla $\rightarrow$ Kod visoko-dimenzionalnih i ne-konveksnih kriterijskih funkcija vrlo su česti i predstavljaju problem u optimizaciji; vrlo je mali gradijent

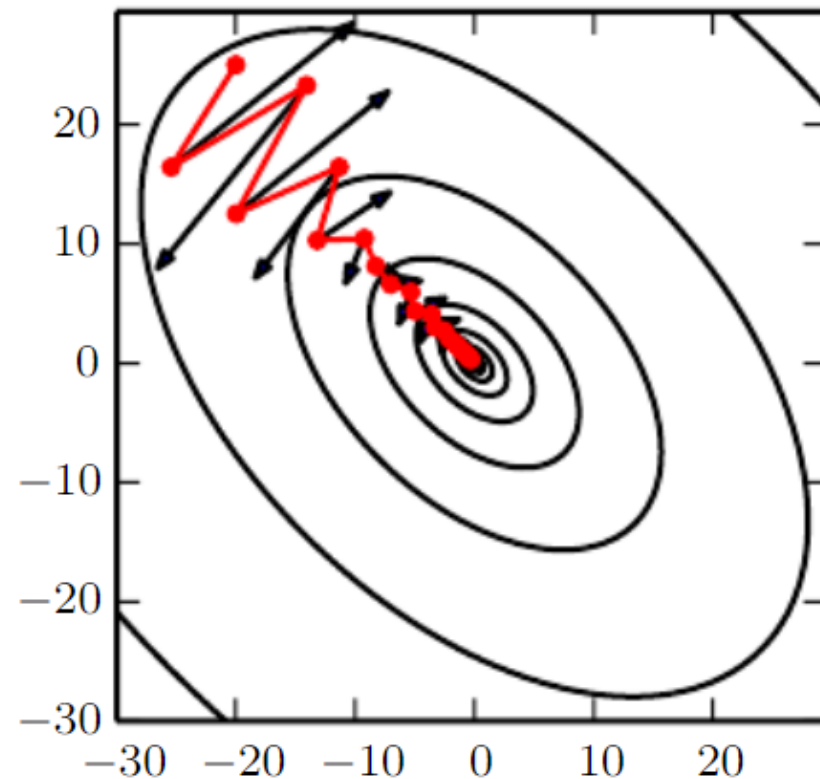
„Intuitivno, u slučaju velikog broja dimenzija, šansa da svi smjerovi oko kritične točke vode prema gore (pozitivna zakrivljenost) eksponencijalno je mala s obzirom na broj dimenzija, osim ako kritična točka nije globalni minimum ili je negdje na razini pogreške blizu nje.”

SGD s momentom

- Vrlo česti pristup kod dubokog učenja → SGD s momentom
- Smjer promjene parametara nije definiran samo gradijentom već „momentom – brzinom v “ odnosno eksponencijalnim usrednjavanjem prethodnih gradijenata (ovime se filtriraju oscilacije → brža konvergencija)
- Na početku procesa učenja moguće je imati jako velike gradijente pa je vrijednost parametra γ manja (npr. 0.5), dok je u kasnijim iteracija moguće koristiti veću vrijednost (0.9)

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$

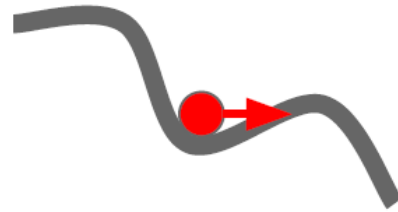
$$\theta = \theta - v_t$$



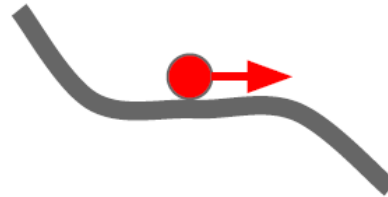
SGD s momentom

- Razvija se odgovarajuća „brzina“ tijekom vremena (prebaci se minimum)

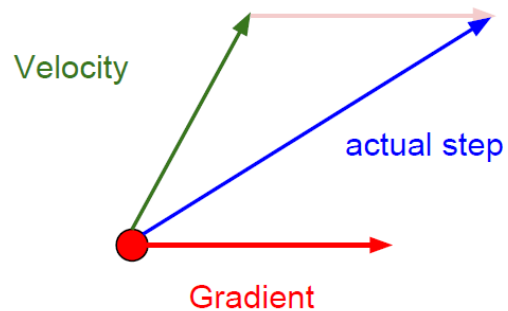
Local Minima



Saddle points



Momentum update:



$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$

$$\theta = \theta - v_t$$

Adagrad and RMSprop

- Stopa učenja se prilagođava prema parametrima → parametri koji se često osvježavaju (povezani sa značajkama koje se često pojavljuju) imaju nižu stopu učenja i obrnuto
- Korisno kada imamo rijetke podatke
- Otprilike isti "napredak" u svim dimenzijama
- Zbroj kvadrata gradijenata prati se po svakom parametru; stopa učenja najčešće je jednaka 0,01

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}.$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t.$$

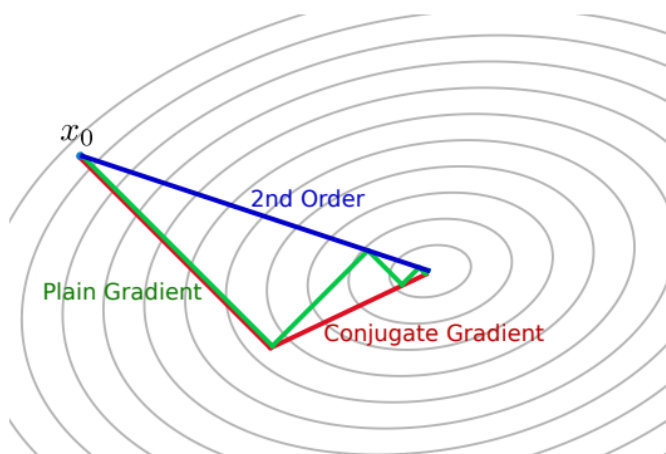
Adagrad and RMSprop

- Tijekom vremena, stopa učenja se smanjuje → dobro za konveksne probleme
- Međutim, za duboke mreže, praktičnija je modifikacija nazvana RMSprop → ponderirani zbroj prošlih gradijenata

$$E[g^2]_t = 0.9E[g^2]_{t-1} + 0.1g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Ostali postupci optimizacije...

- **Adadelta, Adam, Nadam, AMSGrad, AdaMax...**
- Jedno od rješenja problema 1 je uporaba optimizacijskih algoritama zasnovanih na drugoj derivaciji (npr. Newtonova metoda) i modifikacija takvih metoda za rješavanje problema sedla
- Međutim, metode zasnovane na drugoj derivaciji ne koriste se za duboke mreže i velike skupove podataka zbog velikih računalnih zahtjeva



$$\theta^* = \theta_0 - H^{-1} \nabla_{\theta} J(\theta_0)$$

Izravni skok na minimalnu
aproksimaciju lokalnog
kvadrata (bez stope učenja)

Optimizatori...

- Optimizator je jedan od dva argumenta potrebna za sastavljanje Keras modela; možete ga koristiti na dva načina

```
from tensorflow import keras
from tensorflow.keras import layers

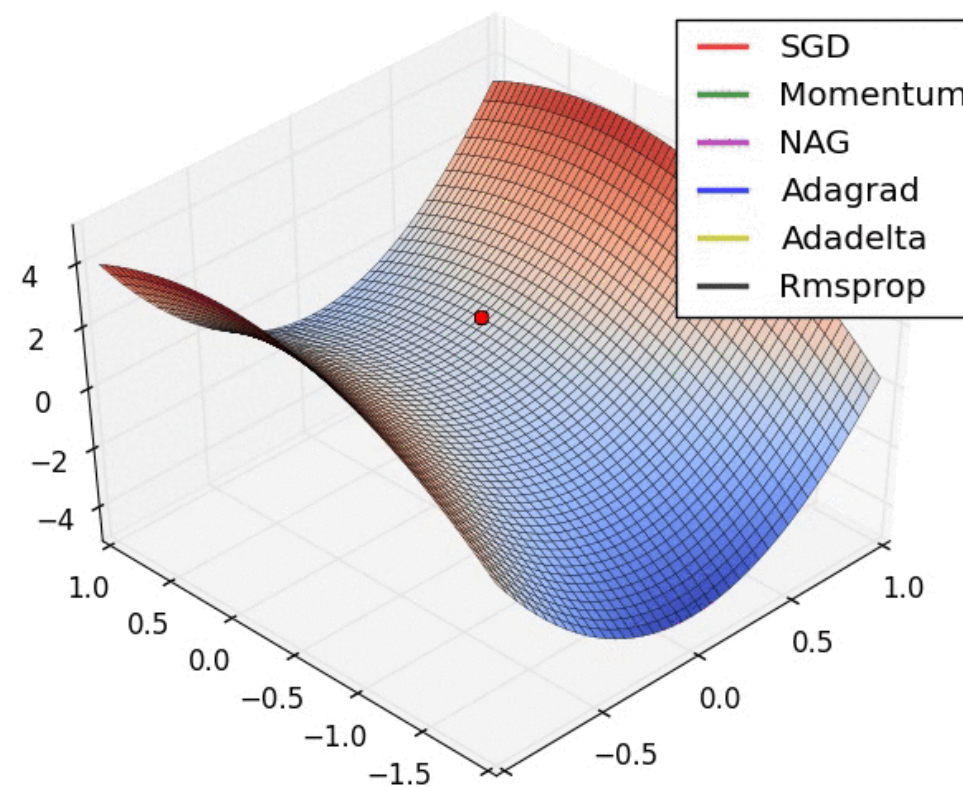
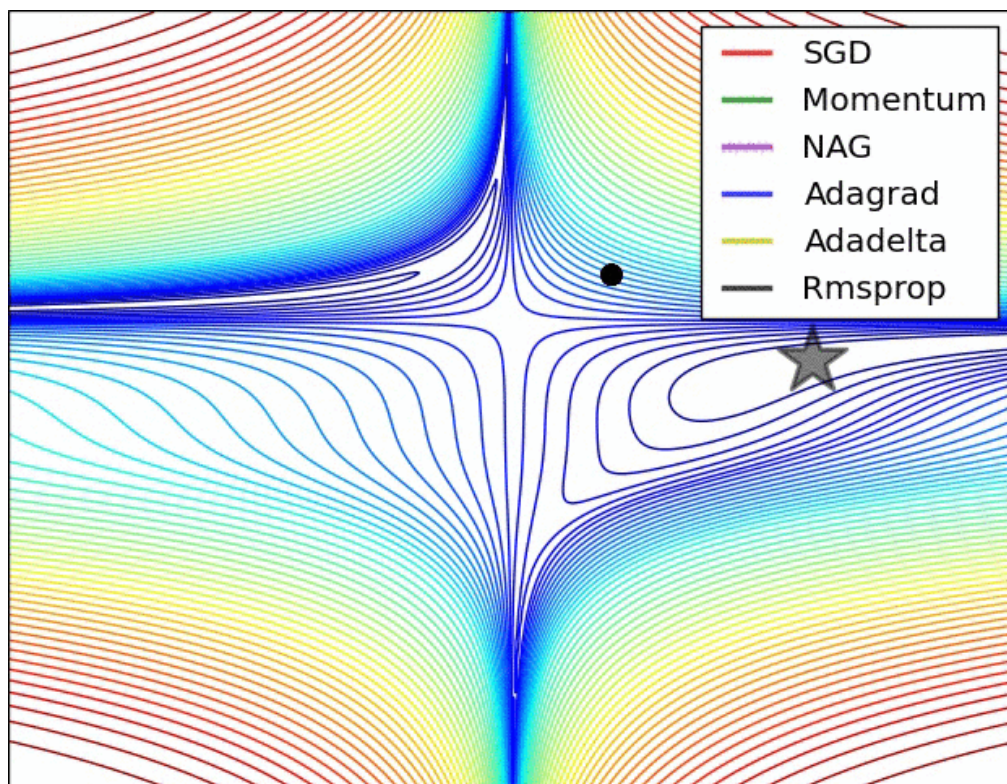
model = keras.Sequential()
model.add(layers.Dense(64, kernel_initializer='uniform', input_shape=(10,)))
model.add(layers.Activation('softmax'))

opt = keras.optimizers.Adam(learning_rate=0.01)
model.compile(loss='categorical_crossentropy', optimizer=opt)
```

```
# pass optimizer by name: default parameters will be used
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

SGD i modifikacije - vizualizacija

- Ponašanje blizu sedla...



Veličina *batcha* (batch size)

Dominic Masters, Carlo Luschi, [Revisiting Small Batch Training for Deep Neural Networks](#), 2018.

The collected experimental results for the CIFAR-10, CIFAR-100 and ImageNet datasets show that increasing the mini-batch size progressively reduces the range of learning rates that provide stable convergence and acceptable test performance. On the other hand, small mini-batch sizes provide more up-to-date gradient calculations, which yields more stable and reliable training.

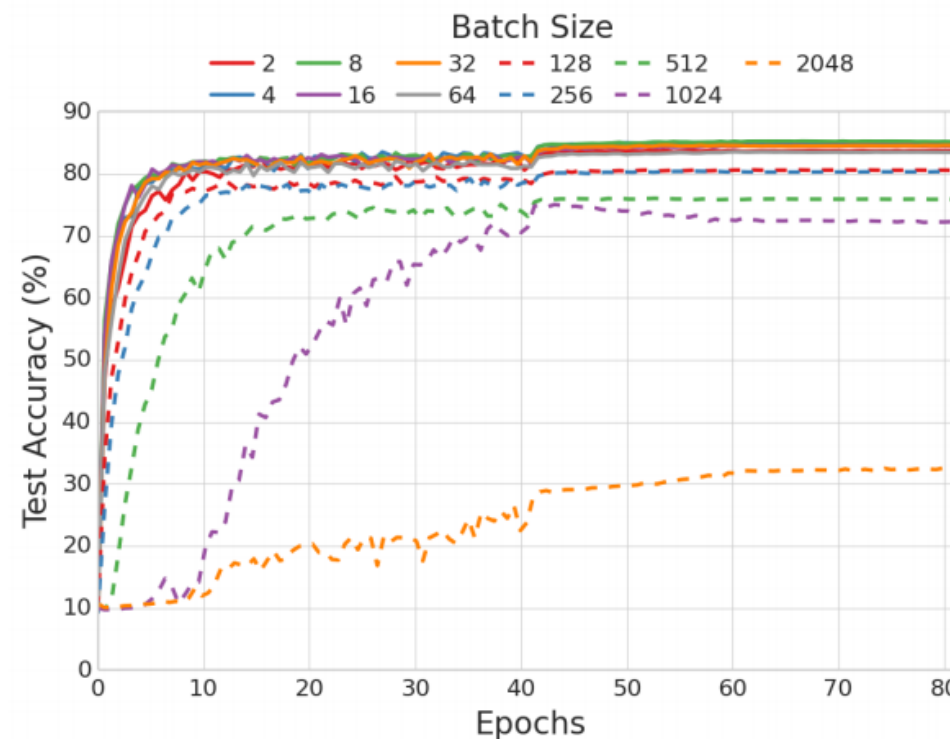


Figure 4: Convergence curves for ResNet-32 model with BN, for $\tilde{\eta} = 2^{-8}$ and for different values of the batch size. CIFAR-10 dataset without data augmentation.

Batch normalization...ubrzavanje procesa učenja

- Sergey Ioffe, Christian Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, 2015.
- Tijekom učenja mreže, distribucija ulaza svakog sloja se mijenja jer se mijenjaju parametri prethodnih slojeva nakon svakog *mini batcha* → ovo usporava proces učenja (potrebno je koristiti manju stopu učenja i pažljivo inicijalizirati parametre mreže)
- **Rješenje: normalizacija ulaza pojedinog sloja tijekom učenja (podaci imaju srednju vrijednost 0 i standardnu devijaciju jednaku 1)**

*“Applied to a state-of-the-art image classification model, Batch Normalization **achieves the same accuracy with 14 times fewer training steps**, and beats the original model by a significant margin.”*

Very deep models involve the composition of several functions or layers. The gradient tells how to update each parameter, under the assumption that the other layers do not change. In practice, we update all of the layers simultaneously.

Batch normalization...ubrzavanje procesa učenja

- Tijekom učenja mreže, distribucija ulaza svakog sloja se mijenja jer se mijenjaju parametri prethodnih slojeva nakon svakog *mini batcha* → ovo usporava proces učenja (potrebno je koristiti manju stopu učenja i pažljivo inicijalizirati parametre mreže)
- **Rješenje**: normalizacija ulaza pojedinog sloja tijekom učenja (podaci imaju srednju vrijednost 0 i standardnu devijaciju jednaku 1)
- Obično se umeće ispred potpuno povezanog sloja, ili konvolucijskog sloja, a prije nelinearnosti

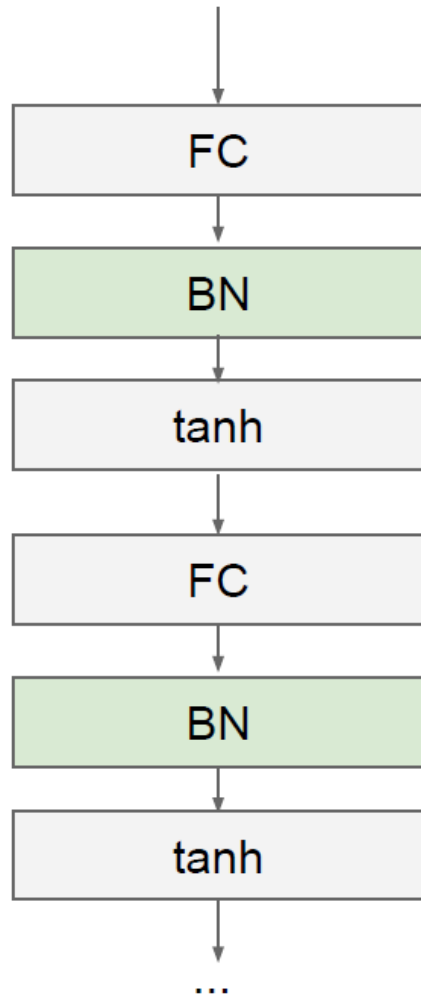
Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Parametri skale i pomaka uče se korištenjem *backpropagation* algoritma

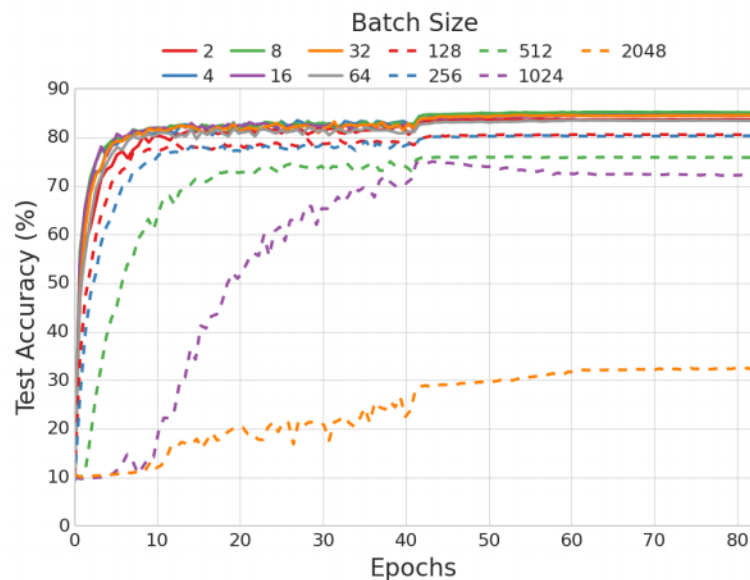
Batch normalization...ubrzavanje procesa učenja



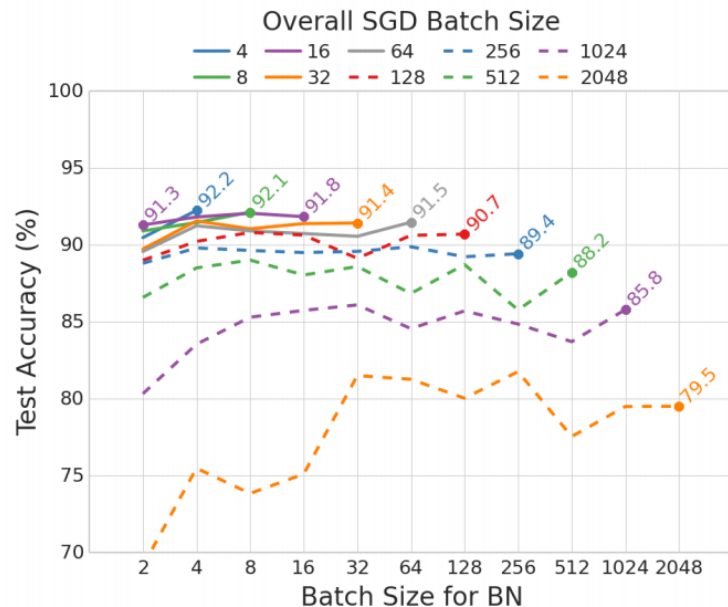
- Pospješuje tok gradijenata kroz mrežu (stabilnost učenja)
- Omogućuje korištenje većeg *learning rate*-a (ubrzava učenje)
- Postupak učenja robusniji na lošu inicijalizaciju
- Može se shvatiti i kao „slaba” regularizacija (unosí šum u aktivacije svakog sloja) → sprječava *overfitting*

Batch size s uključenom batch normalizacijom

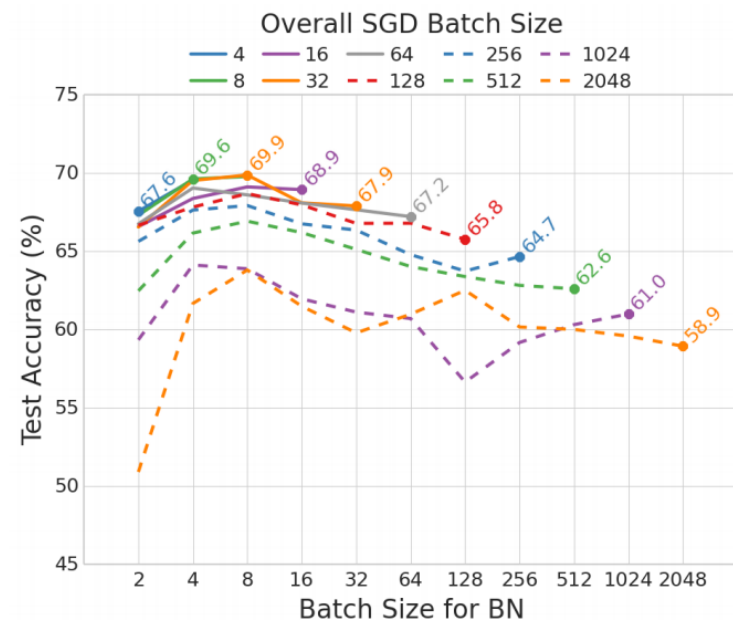
Dominic Masters, Carlo Luschi, [Revisiting Small Batch Training for Deep Neural Networks](#), 2018.



**Bez batch
normalizacije**



(a) CIFAR-10



(b) CIFAR-100

Figure 13: Test performance of ResNet-32 model with BN, for different values of the overall SGD batch size and the batch size for BN. CIFAR-10 and CIFAR-100 datasets with data augmentation.

Batch normalization...ubrzavanje procesa učenja

- BatchNormalization layer u Kerasu

BatchNormalization class

```
tf.keras.layers.BatchNormalization(  
    axis=-1,  
    momentum=0.99,  
    epsilon=0.001,  
    center=True,  
    scale=True,  
    beta_initializer="zeros",  
    gamma_initializer="ones",  
    moving_mean_initializer="zeros",  
    moving_variance_initializer="ones",  
    beta_regularizer=None,  
    gamma_regularizer=None,  
    beta_constraint=None,  
    gamma_constraint=None,  
    synchronized=False,  
    **kwargs  
)
```


Regularizacija

- Kako unaprijediti performanse modela na „neviđenim“ podacima?
- Neuronske mreže imaju velik broj parametara (najčešće milijune), skup podataka je obično značajno manji
- Spriječiti pretjerano usklađivanje na podatke za učenje moguće je primjenom regularizacije:
 - **L2 regularizacija**
 - **L1 regularizacija**
 - **Dropout**

L2 regularizacija

- L2 regularizacija je najčešći tip regularizacije (često se naziva i *weight decay*)
- Ideja – sankcionirati ekstremne vrijednosti parametara (težina)
- Kriterijska funkcija može se regulirati dodavanjem jednostavnog regularizacijskog člana, koji će smanjiti težine tijekom treniranja modela

$$w^* \leftarrow \arg \min_w \sum_{(x,y) \subseteq (X,Y)} \mathcal{L}(y, a_L(x; w_1, \dots, w_L)) + \frac{\lambda}{2} \sum_l \|w_l\|^2$$

- Preko **parametra regularizacije**, λ obično iznosi oko 0.01, može se kontrolirati koliko će se dobro model naučiti na trening podacima, zadržavajući male težine
- Povećanjem vrijednosti λ povećavamo snagu regulacije

L2 regularizacija

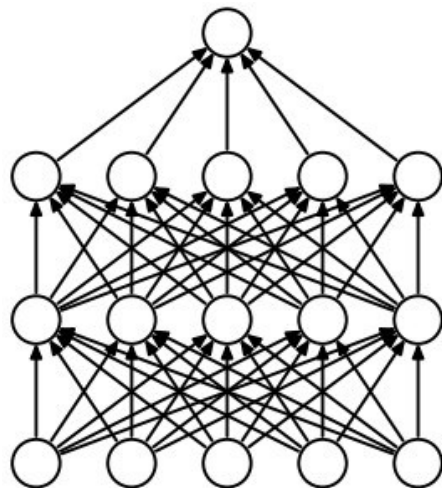
- L2 regularizacija predstavlja jedan od pristupa za smanjivanje složenosti modela kažnjavanjem velikih pojedinačnih težina (koje bi predstavljale preprilagođavanje na trening podatke)

$$w^* \leftarrow \arg \min_w \sum_{(x,y) \in (X,Y)} \mathcal{L}(y, a_L(x; w_1, \dots, w_L)) + \frac{\lambda}{2} \sum_l \|w_l\|^2$$

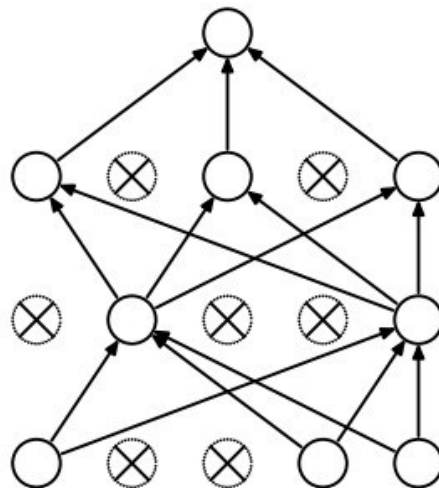
- Regularizacija je još jedan razlog zašto je važno skaliranje značajki (atributa) kao što je standardizacija
- **Kako bi regularizacija ispravno funkcionirala, moramo osigurati da su sve naše značajke na usporedivim skalama**

Dropout (Izostavljanje)

- N. Srivastava et al. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, 2014.
- Vrlo popularna, efikasna i jednostavna tehnika regularizacije (dubokih) neuronskih mreža za izbjegavanje *overfittinga* i poboljšanje performansi generalizacije
- Ključna ideja je nasumično ispuštanje neurona (zajedno s njihovim vezama) iz neuronske mreže tijekom treninga



(a) Standard Neural Net



(b) After applying dropout.

- Tipično je vjerojatnost izbacivanja neurona jednaka 0.5
- Znatno ubrzanje
- Dodaje se slučajna varijabla koja sprječava pretjerano usklađivanje tijekom učenja
- Robusniji neuroni, sprječava ko-adaptaciju (*node interactions*)

Dropout (Izostavljanje)

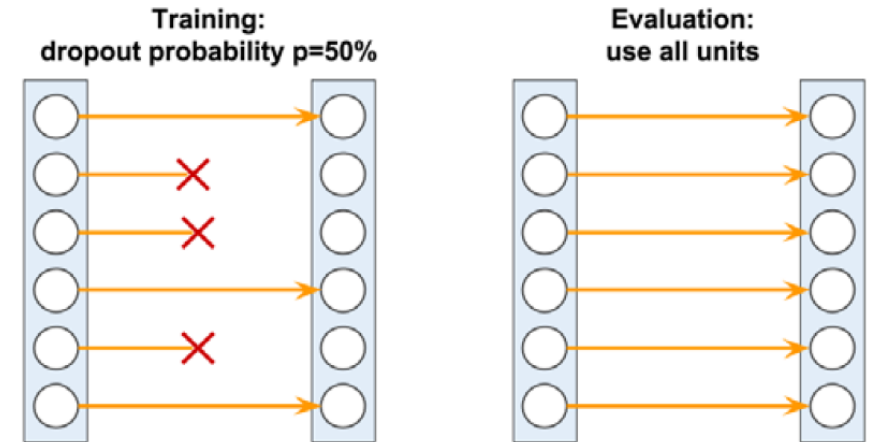
- Dropout se obično primjenjuje u višim skrivenim slojevima
- Tokom faze treninga neuronske mreže dio neurona skrivenih slojeva se nasumično izostavlja u svakoj iteraciji s vjerojatnošću p_{drop}
- Kada se odbaci određeni dio neurona, težine povezane s preostalim neuronima ponovno se mijenjaju kako bi se uzeli u obzir nedostajući (ispušteni) neuroni
- Efekt ovog nasumičnog izostavljanja je što je mreža primorana učiti suvišnu reprezentaciju podataka
- Prema tome, mreža se ne može oslanjati na aktivaciju bilo kojeg skupa skrivenih jedinica jer su one možda isključene u neko vrijeme tokom treniranja, i **primorana je učiti više osnovnih i robusnijih obrazaca iz podataka**

Dropout (Izostavljanje)

- Dio neurona su nasumično neaktivni (izostavljeni neuroni su odabrani nasumično u svakom prosljeđivanju treninga)
 - Međutim, tokom predikcije/evaluacije svi neuroni će učestvovati u izračunavanju preaktivacije sljedećeg sloja
-
- *Dropout* klasa u Kerasu

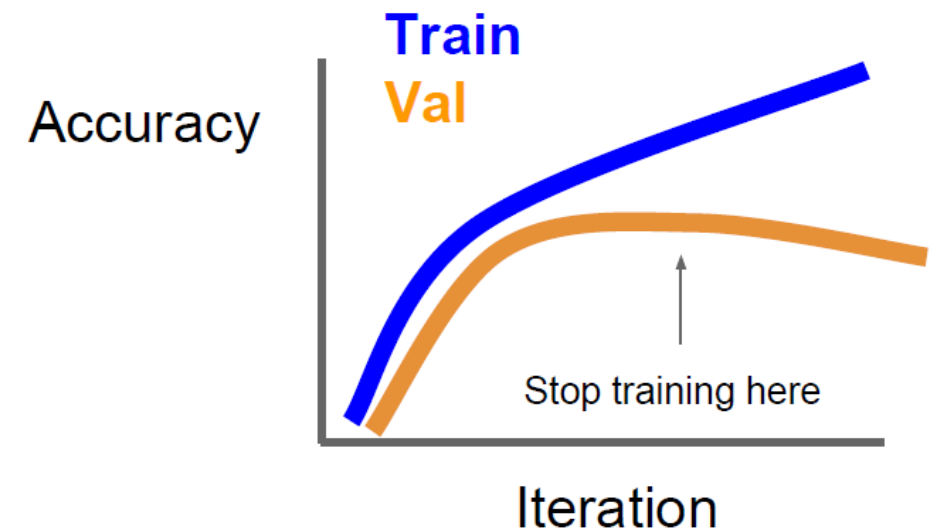
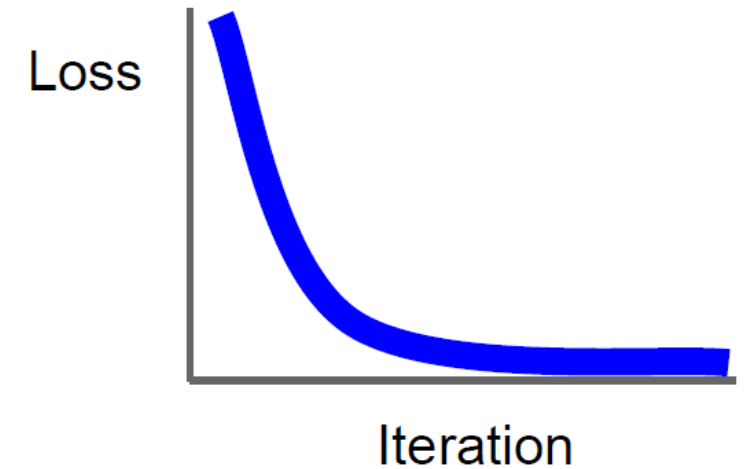
Dropout class

```
tf.keras.layers.Dropout(rate, noise_shape=None, seed=None, **kwargs)
```



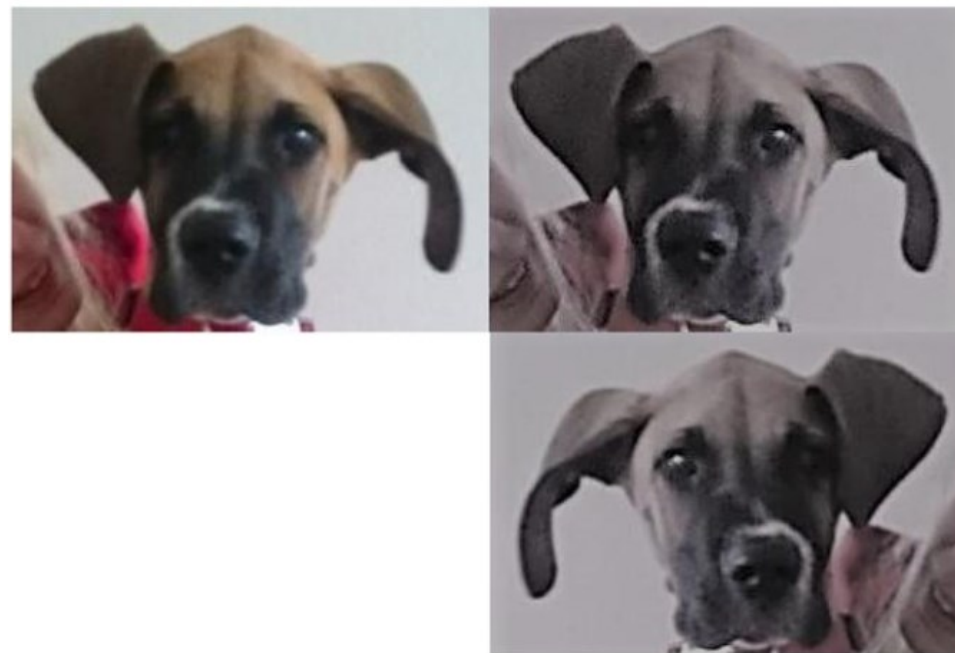
Rano zaustavljanje

- Drugi princip sprječavanja *overfittinga*
- Prate se performanse modela na zasebnom, validacijskom skupu
- Učenjem mreže smanjuje se pogreška na skupu za učenje kao i na validacijskom skupu (obično nešto sporije); tj. raste preciznost
- **Zaustaviti proces učenja kada pogreška na validacijskom skupu počinje rasti** (pretpostavljamo da mreža počinje *overfittati* skup za učenje)
- Ili učiti mrežu duže vrijeme, ali spremati „slike“ modela (kako bi mogli izvući onaj koji je imao najmanju pogrešku na validacijskom skupu)



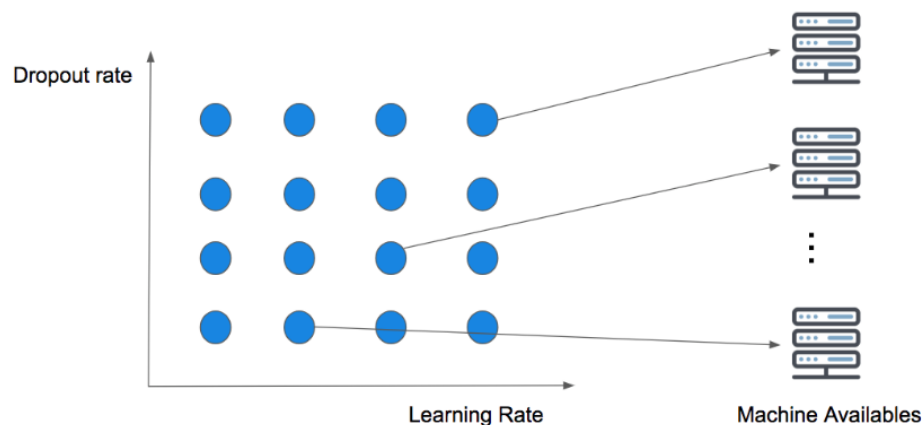
Augmentacija skupa za učenje

- Jedan od načina sprječavanja *overfittinga*; poboljšava generalizacijske sposobnosti mreže
- Podrazumijeva nasumične transformacije slika iz skupa za učenje pri čemu se zadržava oznaka (labela) slike:
 - Horizontal flip
 - Crop
 - Rotacije
 - Translacije
 - Uvećavanje
 - Mijenjanje osvjetljenja, kontrasta ili boje
 - ...
- Transformacije kakve bi se mogle pojaviti u praksi



Proces odabira hiperparametara mreže

- Potrebno je u praksi podesiti tj. prilagoditi hiperparametre povezane s procesom učenja (stopa učenja, dropout, batch size, regularizacijski parametar...) → iterativni proces
- U prvoj fazi učenja provjerite kako se proces učenja otprilike ponaša s određenim vrijednostima hiperparametara (mijenjajte parametre u većim koracima)
- Kasnije finije prilagodite hiperparametre
- Problem: Učenje može potrajati za svaku kombinaciju hiperparametara



Grid Search on two variables in a parallel concurrent execution

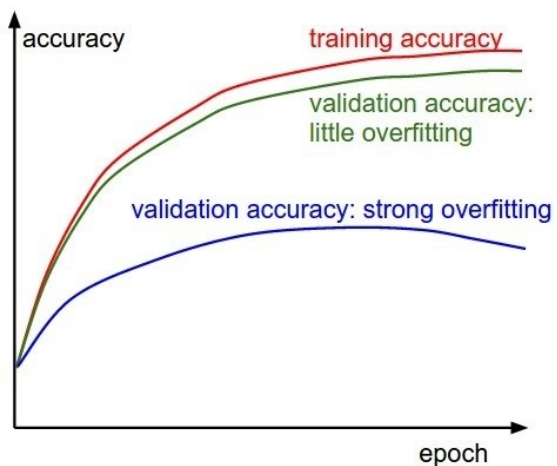
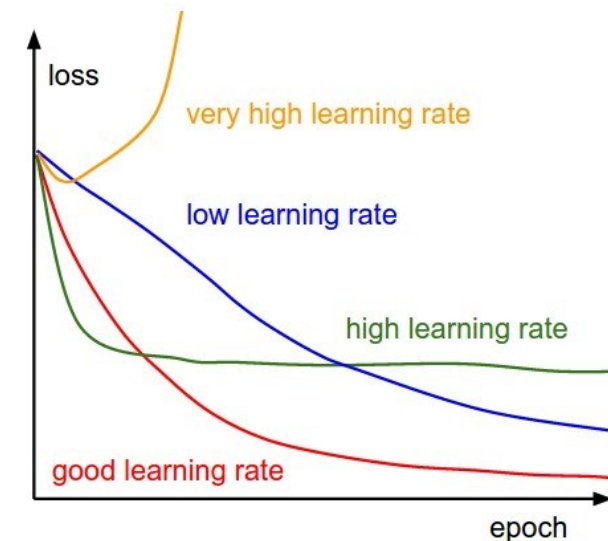
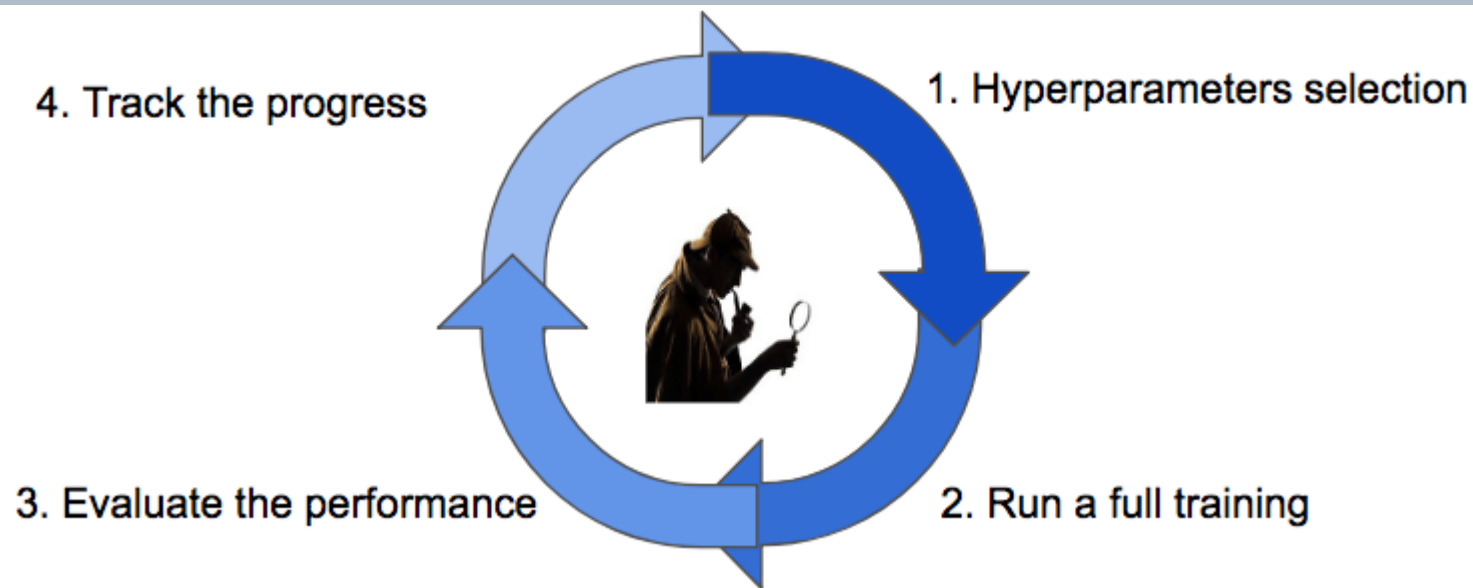
Problem ako imamo veći broj parametara (više od 3-4)

Broj kombinacija brzo raste

U ovom slučaju upotrijebite nasumično pretraživanje – pretpostavka je da neki od hiperparametara nisu bitni

Više informacija: [Random Search for Hyper-Parameter Optimization, 2012.](#)

Tijek procesa učenja



Veliki „razmak” između train i val preciznosti → overfitting

Nema „razmaka” između train i val preciznosti → možebitna premala složenost modela

P i t a n j a ?

Pozivne funkcije (*Callbacks*) u Kerasu

Pozivne funkcije (*Callbacks*) u Kerasu

- Pozivna funkcija općenito je neka vrsta izvršnog koda koji se prosljeđuje kao argument drugom kodu, a od drugog koda se očekuje da pozove dani kod u određeno vrijeme
- ***Keras callback*** je objekt koji se aktivira tijekom učenja (npr. na početku ili na kraju epohe, nakon jednog *batcha*...)
- Postoje gotovi *callbacks*, ali možete kreirati i vlastiti *callback*
- *Callbacks* se prosljeđuju metodi **.fit()** u obliku liste koja sadrži sve *callbacks* koje želimo primijeniti tijekom procesa učenja
- Mogu se koristiti za razne korisne radnje: povremeno spremanje modela na disk, rano zaustavljanje, pisanje u TensorBoarda *loggove* za praćenje napretka učenja, podešavanje stope učenja...

ModelCheckpoint

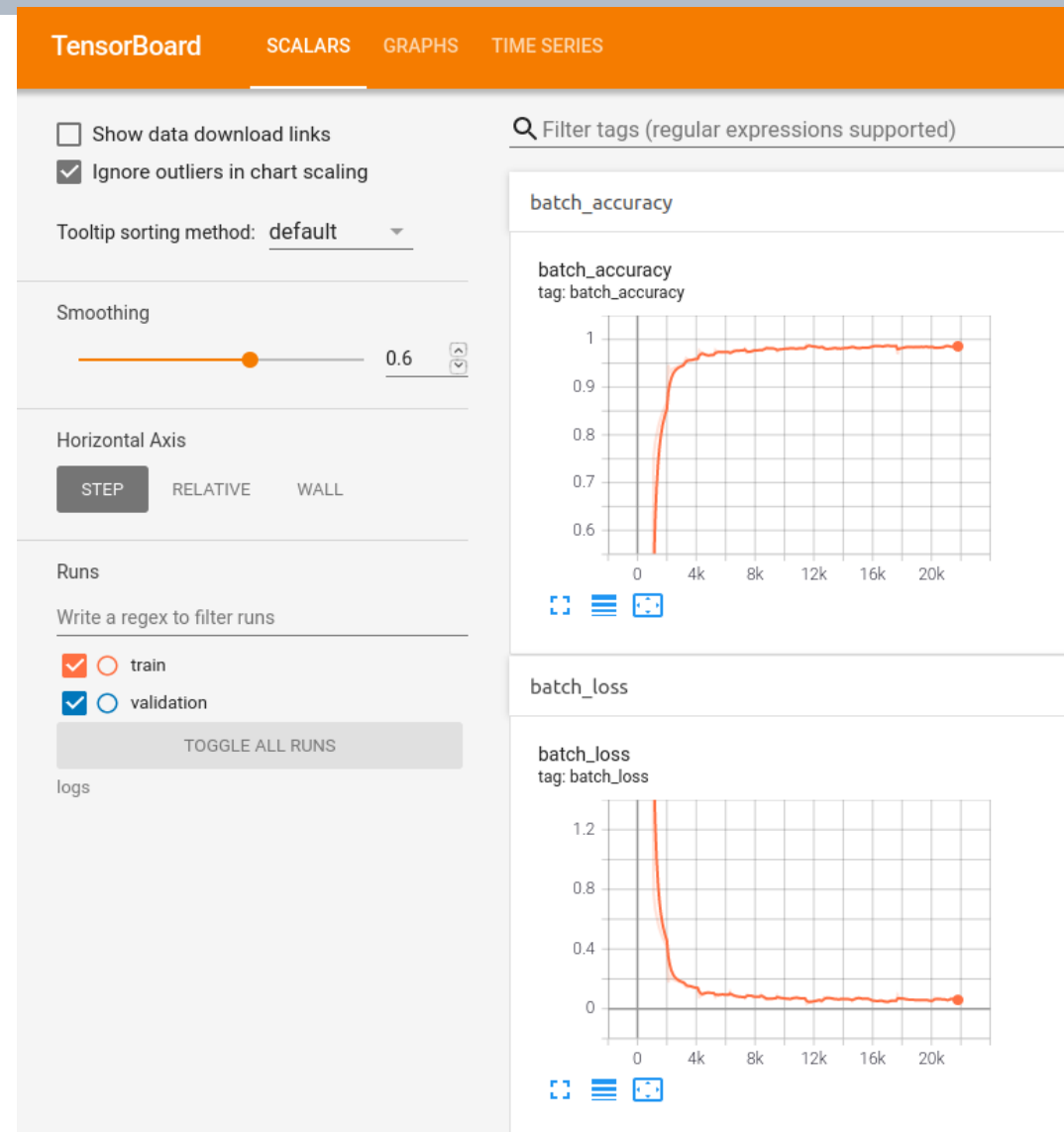
- ModelCheckpointing je *callback* koji sprema model na disk pod određenim uvjetima
- Neki argumenti:
- **filepath**: putanja za spremanje datoteke modela
- **save_best_only**: ako je True, sprema se samo kada se model smatra "najboljim" i posljednji najbolji model prema metrici koja se prati neće biti prebrisan
- **monitor**: naziv metrike koja se prati (npr. val_loss)
- **mode**: jedan od {'auto', 'min', 'max'}; odluka o prepisivanju trenutnog modela donosi se na temelju maksimiziranja ili minimiziranja nadzirane metrike
- **save_freq**: epoch ili cijeli broj. Kada koristite 'epoch', *callback* sprema model nakon svake epohe. Kada se koristi cijeli broj, *callback* sprema model na kraju tog broja *batcheva*

ModelCheckpoint class

```
tf.keras.callbacks.ModelCheckpoint(  
    filepath,  
    monitor: str = "val_loss",  
    verbose: int = 0,  
    save_best_only: bool = False,  
    save_weights_only: bool = False,  
    mode: str = "auto",  
    save_freq="epoch",  
    options=None,  
    initial_value_threshold=None,  
    **kwargs  
)
```

TensorBoard

- TensorBoard je alat koji omogućava vizualizaciju tijeka procesa učenja modela
 - Kriterijska funkcija (*loss*) i točnost, razni drugi grafovi
- Tijekom učenja potrebno je bilježiti određene zapise (*loggove*) kojima se zatim pristupa putem web preglednika
- Keras → pozvati odgovarajući *callback* **TensorBoard**



TensorBoard

Važni argumenti:

- **log_dir**: putanja do direktorija gdje se spremaju zapisi koje će TensorBoard parsirati
- **update_freq**: 'batch' ili 'epoch' ili integer.
 - Kada se koristi 'epoch'/'batch', zapisuju se iznosi *loss* i željenih metrika u TensorBoard nakon svake epohe/*batch*-a.
 - Ako se koristi integer **x**, tada će se *loss* i ostale metrike zapisivati svaki **x** *batcheva*

TensorBoard class

```
tf.keras.callbacks.TensorBoard(  
    log_dir="logs",  
    histogram_freq=0,  
    write_graph=True,  
    write_images=False,  
    write_steps_per_second=False,  
    update_freq="epoch",  
    profile_batch=0,  
    embeddings_freq=0,  
    embeddings_metadata=None,  
    **kwargs  
)
```


TensorBoard - primjer

- Prvo se definira *callback* i proslijedi se `.fit()` metodi

```
keras.callbacks.TensorBoard(log_dir = 'logs/dropout_bn_lr',  
                             update_freq = 100),
```

- Pokrene se Tensorboard iz upravljačke linije

```
tensorboard --logdir=path_to_your_logs
```

- Pokrene se web preglednik i odebere se:

```
http://localhost:6006/
```

LearningRateScheduler

- Na početku svake epohe ovaj *callback* dobiva ažuriranu vrijednost stope učenja iz funkcije rasporeda s trenutnom epohom i trenutnom stopom učenja i primjenjuje ažuriranu stopu učenja na optimizatoru

LearningRateScheduler class

```
tf.keras.callbacks.LearningRateScheduler(schedule, verbose=0)
```

- **schedule**: funkcija koja uzima indeks epohe (cijeli broj, indeksiran od 0) i trenutnu stopu učenja (float) kao ulaze i vraća novu stopu učenja kao izlaz (float)

ReduceLROnPlateau

- Smanjuje stopu učenja kada se metrika prestane poboljšavati
- Neki argumenti:
- **monitor**: metrika koju treba pratiti.
- **factor**: faktor za koji će se smanjiti stopa učenja
- **patience**: broj epoha bez poboljšanja nakon kojih će se stopa učenja smanjiti
- **cooldown**: broj epoha za čekanje prije nastavka normalnog rada nakon što je stopa učenja smanjena

ReduceLROnPlateau class

```
tf.keras.callbacks.ReduceLROnPlateau(  
    monitor="val_loss",  
    factor=0.1,  
    patience=10,  
    verbose=0,  
    mode="auto",  
    min_delta=0.0001,  
    cooldown=0,  
    min_lr=0,  
    **kwargs  
)
```

EarlyStopping

- Prestanak treninga kada se nadzirana metrika prestane poboljšavati
- Neki argumenti:
- **monitor**: metrika koju treba pratiti
- **verbose**: mod 0 ili 1. Mod 0 je tih, a mod 1 prikazuje poruke kada se *callback* aktivira
- **patience**: broj epoha bez poboljšanja nakon kojih će se trening prekinuti.
- **mode**: jedan od {"auto", "min", "max"}

EarlyStopping class

```
tf.keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    min_delta=0,  
    patience=0,  
    verbose=0,  
    mode="auto",  
    baseline=None,  
    restore_best_weights=False,  
    start_from_epoch=0,  
)
```

Data augmentation

- Augmentacija – metoda povećanja količine trening podataka dodavanjem (blago) modificiranih kopija već postojećih podataka
 - Pomaže u smanjenju *overfittinga*
- Keras nudi klasu **ImageDataGenerator** koja omogućuje vrlo jednostavnu implementaciju augmentacije u stvarnom vremenu
 - Rotacija
 - Translacija
 - „Flipanje”
 - Promjena svjetline
 - Zumiranje
- Objekt generira serije slika

Data augmentation - primjer

- Primjena augmentacije na numpy polje
- Primijetite da augmentiramo samo trening skup slika

```
17 train_datagen = ImageDataGenerator(  
18     rescale = 1./255,  
19     rotation_range = 15,  
20     width_shift_range = 0.1,  
21     height_shift_range = 0.1,  
22     zoom_range = 0.2,  
23     horizontal_flip = True,  
24     fill_mode='nearest')  
25  
26 val_datagen = ImageDataGenerator(rescale=1./255)  
27  
28 test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_iter = train_datagen.flow(X_train, y_train, batch_size = batch_size)  
valid_iter = val_datagen.flow(X_val, y_val, batch_size = 1)  
test_iter = test_datagen.flow(X_test, y_test, batch_size = 1)
```

Ovo prihvaća Keras model .fit() metoda

Data augmentation

- Neki argumenti:
- **rotation_range**: Int; Raspon stupnjeva za nasumične rotacije
- **brightness_range**: Tuple ili lista od dva float-a. Raspon za odabir vrijednosti promjene svjetline
- **zoom_range**: Float ili [lower, upper]. Raspon za nasumično zumiranje. Ako je float, [lower, upper] = [1-zoom_range, 1+ zoom_range]
- **horizontal_flip**: Boolean. Nasumično „flipa” ulaze horizontalno
- **vertical_flip**: Boolean. Nasumično „flipa” ulaze vertikalno
- **rescale**: faktor promjene skale. Default je None. Ako je None ili 0, ne primjenjuje se reskaliranje, inače podatke množimo s navedenom vrijednošću (nakon primjene svih ostalih transformacija)
- **preprocessing_function**: funkcija koja će se primijeniti na svaki ulaz. Funkcija će se pokrenuti nakon što se slika reskalira i augmentira. Funkcija bi trebala uzeti jedan argument: jednu sliku (Numpy tenzor ranga 3) i trebala bi kao izlaz dati Numpy tenzor istog oblika

ImageDataGenerator class

```
tf.keras.preprocessing.image.ImageDataGenerator(  
    featurewise_center=False,  
    samplewise_center=False,  
    featurewise_std_normalization=False,  
    samplewise_std_normalization=False,  
    zca_whitening=False,  
    zca_epsilon=1e-06,  
    rotation_range=0,  
    width_shift_range=0.0,  
    height_shift_range=0.0,  
    brightness_range=None,  
    shear_range=0.0,  
    zoom_range=0.0,  
    channel_shift_range=0.0,  
    fill_mode="nearest",  
    cval=0.0,  
    horizontal_flip=False,  
    vertical_flip=False,  
    rescale=None,  
    preprocessing_function=None,  
    data_format=None,  
    validation_split=0.0,  
    dtype=None,  
)
```

P i t a n j a ?